

Chapter 0: How to Approach an ML System Design Case

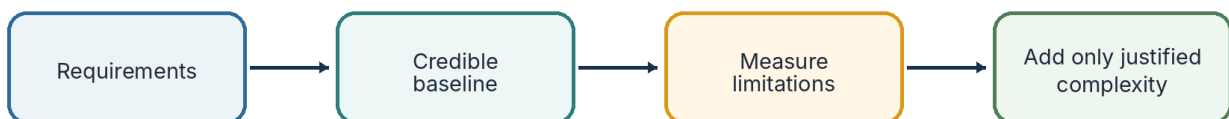
A minimal-scope, alternatives-driven framework for interviews and internal design reviews

PRIMARY REFERENCE

Study foundation

This handbook began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, comparison framework, diagrams, and interpretations are the author's own.

Minimum sufficient design



A component belongs in the design only when it traces back to a requirement or a measured limitation.

Requirement → design decision → expected benefit → metric or evidence

Figure 0.1 - Begin with the minimum sufficient design and add complexity only when evidence justifies it.

CORE PRINCIPLE

Every component needs a reason

Choose the smallest architecture that can satisfy the current requirements. Add granularity or complexity only to solve a specific product, quality, latency, scale, or operational limitation.

0. How to read this framework

The chapter separates source inspiration, confirmed principles, open choices, and starting recommendations.

FROM THE PRIMARY REFERENCE

Source-derived study structure

The case-study topics and many design alternatives were learned from Machine Learning System Design Interview. Public materials should acknowledge the book and should not reproduce its text or figures verbatim.

CONFIRMED PRINCIPLE

Minimal scope first

Related problems may be mentioned, but a separate objective should remain a separate service rather than being forced into one oversized architecture.

OPEN DESIGN DISCUSSION

Context determines the answer

The framework does not declare one universal model. It asks readers to compare alternatives, state assumptions, and identify what evidence would change the decision.

STARTING RECOMMENDATION

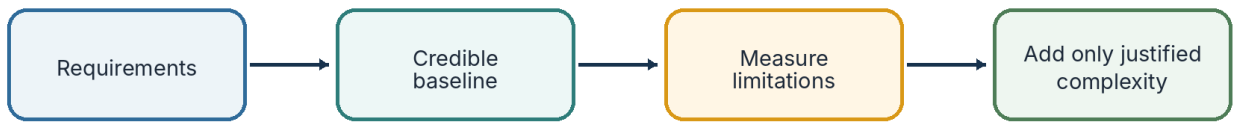
Use a credible baseline

Begin with the easiest viable system that can meet the core task, then let measurable limitations justify the next layer of complexity.

1. Keep the scope minimal

Minimal does not mean weak; it means every component is traceable to a requirement.

Minimum sufficient design



A component belongs in the design only when it traces back to a requirement or a measured limitation.

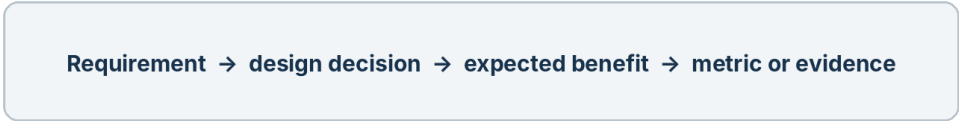


Figure 1.1 - The minimum-sufficient-design loop.

A minimal design may still include multiple stages, multimodal inputs, offline computation, human review, monitoring, and retraining. Those components belong only when the requirements need them.

Question	Why it matters
What is the main service?	Prevents adjacent problems from silently expanding the case.
Which requirement justifies each component?	Makes the reasoning traceable and interview-ready.
What limitation is measured?	Prevents speculative complexity.
What would remove the component?	Encourages simpler production systems when quality is similar.

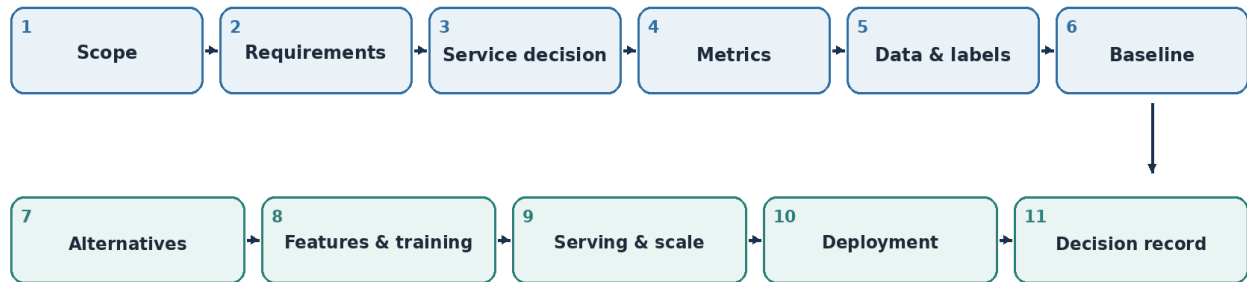
DESIGN DECISION
Main service: _____
Related but separate systems: _____
Out of scope: _____

2. Use a recurring design order

A stable sequence keeps the discussion complete without forcing every case into the same architecture.

Recurring order of an ML system design case

A checklist for completeness, not a rigid script.



Move a section earlier only when a case-specific constraint changes the architecture.

Figure 2.1 - The recurring order used throughout the handbook.

The sequence is a default. Candidate generation may appear before modeling in recommendation systems, while latency may be discussed earlier when it is the dominant constraint.

STARTING RECOMMENDATION

Use the order as a checklist, not a script

Follow the structure until a case-specific constraint requires a different sequence. The objective is completeness and clarity, not rigid formatting.

3. Define scope, requirements, and the service decision

Select the model only after the input, output, product action, and constraints are explicit.

Problem statement

A useful one-sentence objective is: Given X, predict or produce Y so the service can take Z action.

Item	Questions to answer
Input	Who or what receives the prediction? What data is available at request time?
Model output	Score, label, embedding, ranked list, or generated response?
Service action	Recommend, remove, alert, approve, route, or review?
Boundary	Which related systems remain separate?

Functional and non-functional requirements

Functional	Non-functional
What the system must do	Latency and throughput
Supported modalities or languages	Freshness and scale
Granularity of outputs	Reliability and fallback
Human-review or explanation needs	Cost, privacy, and maintainability

Separate the model output from the service action



Examples

- category scores → harmful score → keep / review / remove
- candidate scores → ranking policy → ordered recommendation list
- event probability → threshold policy → alert / no alert

Figure 3.1 - Internal model granularity and the final product action are separate design layers.

DESIGN DECISION
Input: _____
Model output: _____
Service action: _____
Architecture-changing requirement: _____

4. Connect product metrics to offline evaluation

Begin with the business objective, then choose an offline metric that approximates it.

Layer	Example question	Common failure
Business objective	What user or business outcome should improve?	Optimizing a proxy with no product value
Online metric	What numerator, denominator, unit, and time window define success?	Metric growth caused by lower-quality behavior
Offline metric	What can be measured before deployment?	High offline score with weak product lift
Guardrail	What must not degrade?	Improvement achieved through harmful side effects

CONFIRMED PRINCIPLE

Metrics are part of the architecture

Rare positives may favor PR-AUC; ranked outputs may require Precision@K; probability-driven policies may require calibration. The metric should follow the task rather than habit.

DESIGN DECISION

Primary product metric: _____

Offline proxy: _____

Guardrails: _____

Time window and evaluation unit: _____

5. Define data and labels before the model

State exactly what one training example represents and enforce the prediction-time cutoff.

Design item	Required decision
Entities	Users, items, posts, queries, interactions, graphs, reports, timestamps
Training unit	User-candidate pair, query-item pair, post-category pair, or event window
Labels	Positive, negative, unlabeled, weak, trusted, delayed, or multi-label
Time boundary	Only features available at or before prediction time
Missingness	Unknown, intentionally omitted, structurally absent, or delayed

MINIMAL-SCOPE RULE

Do not invent labels the source does not provide

When negatives, attribution windows, or trusted labels are missing, preserve them as open design questions rather than silently filling the gap.

DESIGN DECISION

Training unit: _____
Positive label: _____
Negative or unlabeled case: _____
Trusted benchmark: _____
Temporal cutoff: _____

6. Compare alternatives through three recurring perspectives

The lenses help organize trade-offs without forcing exactly three separate models.

Three recurring perspectives for alternatives

A. Easy to implement

- Fast validation
- Simple debugging
- Credible baseline
- Low serving cost

B. More granular / accurate

- Richer outputs
- Temporal or graph structure
- Cross-feature interactions
- Better difficult-case modeling

C. Production-balanced

- Scalable and maintainable
- Latency and reliability
- Monitoring and fallback
- Quality versus cost

These are decision lenses, not a requirement to invent exactly three separate models.

Figure 6.1 - Easy baseline, richer modeling, and production-balanced design.

Perspective	Primary value	Typical cost
Easy to implement	Fast validation, debugging, and a credible reference	May omit granularity or complex relationships
More granular / accurate	Richer outputs, temporal structure, graph or cross-modal interactions	More data, tuning, compute, and debugging
Industrial preference	Quality balanced with latency, scale, reliability, and maintainability	May deliberately sacrifice peak offline accuracy

CONFIRMED PRINCIPLE

Industrial preference is not automatically the richest model

A production team may keep the baseline, combine simple and rich stages, or run the rich model offline when the quality gain does not justify online complexity.

7. Give every alternative a decision rule

A list of models is not a design discussion until the switching conditions are explicit.

How to compare an alternative

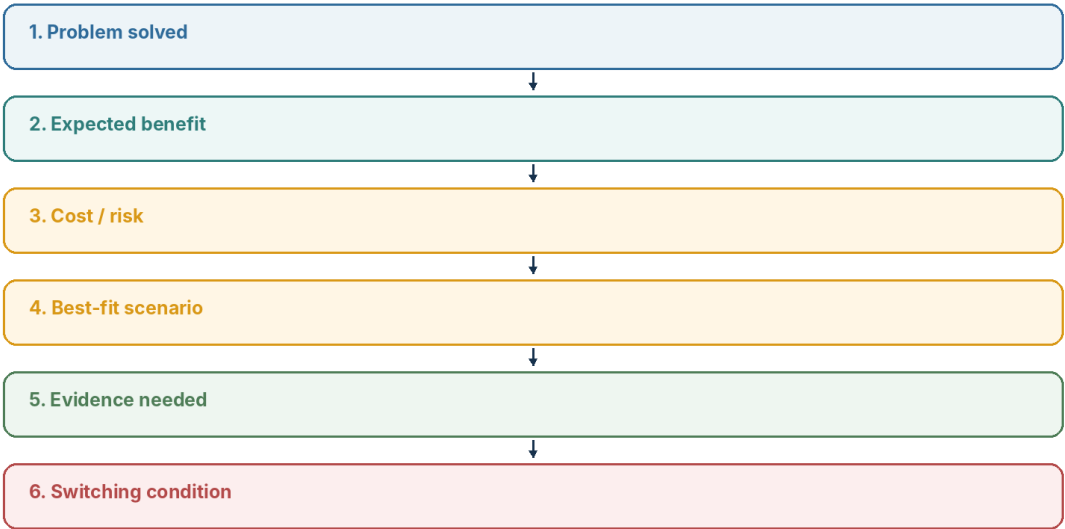


Figure 7.1 - Six questions that turn an option list into a decision.

Option	Problem solved	Benefit	Cost / switch condition
Baseline	Basic task and measurement	Fast, reliable reference	Replace only after a measured failure
Richer model	Specific missed relationship or granularity	Potential quality lift	Keep only if lift justifies data and serving cost
Production-balanced	Operational constraints	Reliable user-facing system	Simplify when quality remains similar

DESIGN DECISION
Selected option: _____
Main reason: _____
Accepted trade-off: _____
Evidence needed: _____
Condition that changes the decision: _____

8. Design features and training around observed problems

Use only features and optimization methods that solve a visible requirement or failure mode.

Feature questions

Question	Reason
Available at prediction time?	Prevents leakage and impossible online behavior
Stable and retrievable?	Prevents training-serving skew
Bias or privacy risk?	Separates predictive value from unacceptable harm
Needed in the main model?	Some features belong only in routing, review, or monitoring

Training method follows the failure mode

Observed problem	Possible design response
Rare positive events	Imbalance-aware metric or loss
Enormous pair space	Negative sampling or candidate narrowing
Tasks dominate shared layers	Task weighting or gradient blending
Weak or noisy labels	Trusted benchmark, filtering, or confidence weighting
Probabilities drive policy	Calibration and category-specific thresholds

STARTING RECOMMENDATION

Start with the simplest loss and split

Add focal loss, class balancing, gradient blending, or specialized sampling only when the data shows the corresponding failure.

9. Design serving and scale from the bottleneck

Separate offline and online work, then use the cheapest stage that preserves the required quality.

A production-balanced design can combine alternatives

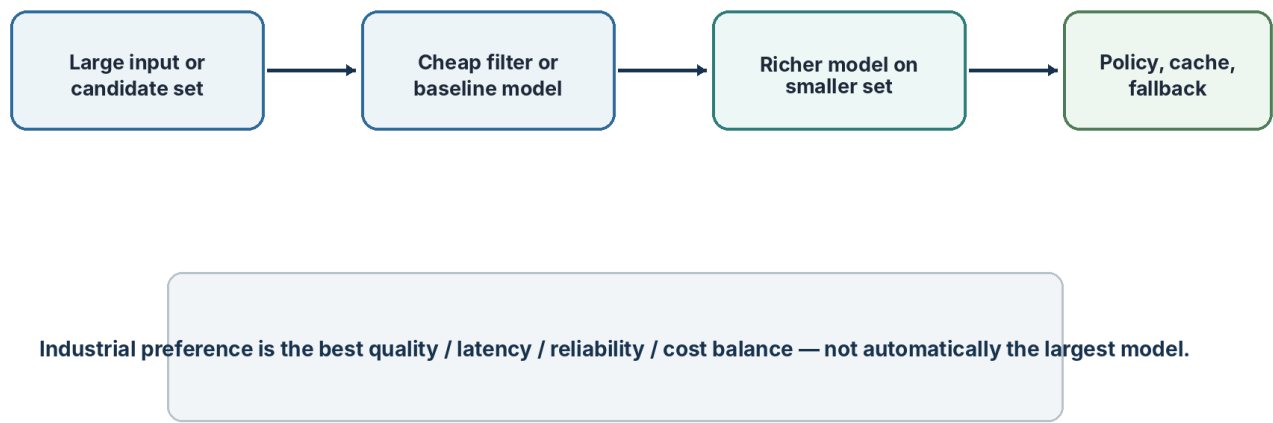


Figure 9.1 - Simple and rich alternatives can be combined into a staged production design.

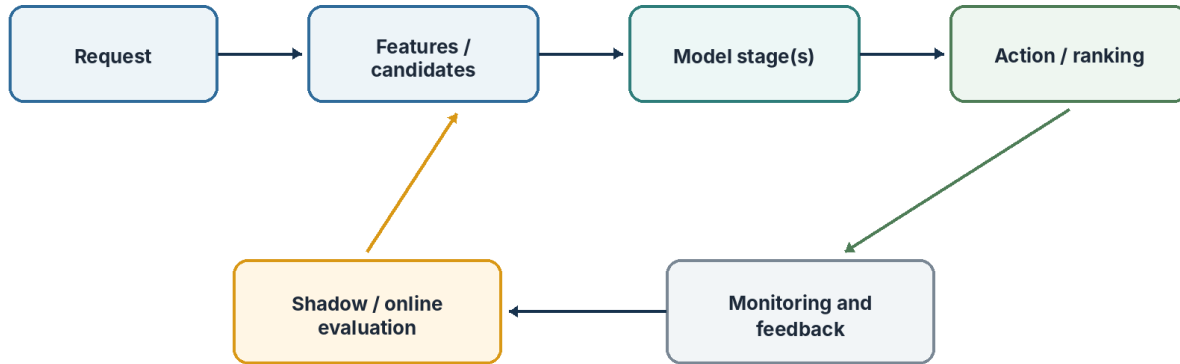
Serving pattern	Best when	Main trade-off
Fully online	Freshness is critical and the model is inexpensive	Highest request-time cost
Precomputed	Outputs change slowly and low latency matters	Staleness
Hybrid	Cached candidates or embeddings need fresh re-ranking	More moving parts, balanced latency

Measured bottleneck	Targeted technique
All-pairs search	Candidate narrowing or approximate retrieval
Long sequence	Approximate or linear attention
Slow online model	Precompute, cache, or lightweight re-ranking
Throughput	Batching or asynchronous processing
Review capacity	Confidence routing and prioritization

10. Validate production risk before broad rollout

Choose shadow deployment, interleaving, or A/B testing according to the question and error cost.

Serving, validation, and monitoring



Use the lowest-risk validation method that can answer the question.

Figure 10.1 - Serving connects to validation, monitoring, and feedback.

Method	Best use	Limitation
Shadow deployment	Safety, latency, score distribution, disagreement review	Cannot measure behavior after real actions
Interleaving	Fast comparison between ranking orders	Narrower than full product impact
A/B testing	Broader causal product impact	Riskier when errors directly affect users

Monitoring and refresh

- Input and feature drift
- Score distributions and calibration
- Category- or segment-level quality
- Latency, errors, and feature availability
- Business outcomes, reports, appeals, or user actions
- Retraining or refresh triggers and rollback conditions

DESIGN DECISION

First validation method: _____

Monitoring metrics: _____

Exit criteria: _____

Rollback condition: _____

11. Use a concise interview flow

A complete answer can remain structured even when time is limited.

Order	Interview move
1	Clarify scope, input, output, and product action
2	State scale, latency, and freshness assumptions
3	Choose product and offline metrics
4	Define data, labels, and temporal cutoff
5	Establish the easiest viable baseline
6	Propose the richer option and the limitation it solves
7	Explain the production-balanced design
8	Describe serving, bottleneck, and fallback
9	Close with deployment, trade-offs, and switching conditions

INTERVIEW-READY STATEMENT

A defensible default

I would begin with the smallest system that satisfies the core requirements, establish a measurable baseline, and add complexity only when a specific product, quality, latency, or scaling limitation justifies it.

12. Avoid common design mistakes

The framework is designed to prevent impressive-sounding but weakly justified architectures.

Mistake	Better reasoning
Start with the model	Start with the service decision and requirements
Put every related problem in scope	State the boundary and design separate services
List alternatives without a rule	Include benefit, cost, evidence, and switching condition
Assume highest accuracy is industrial preference	Balance quality with latency, reliability, and maintenance
Add scaling methods speculatively	Identify the measured bottleneck first
Confuse model output and product action	Add an explicit aggregation or policy layer
Use future data	Enforce prediction-time feature cutoffs

13. Final case checklist

Use this one-page review before publishing or presenting a case.

Area	Check
Scope	Main service, input, output, action, and out-of-scope items are explicit
Requirements	Functional and non-functional constraints are challengeable
Metrics	Product metric, offline proxy, guardrails, unit, and time window are clear
Data	Training unit, labels, trusted benchmark, missingness, and time cutoff are defined
Alternatives	Credible baseline, richer option, and production balance are compared
Training	Each method solves an observed problem
Serving	Offline/online split, bottleneck, cache, and fallback are explicit
Deployment	Validation, monitoring, refresh, and rollback are defined
Decision	Trade-offs, assumptions, evidence, and switching conditions are recorded

DESIGN DECISION

Chosen design: _____

Why it is minimum sufficient: _____

Accepted trade-offs: _____

Evidence still needed: _____

Conditions that change the design: _____

14. Reference and attribution

Public chapters should clearly distinguish the primary study source from original transformations and commentary.

PRIMARY REFERENCE

Machine Learning System Design Interview

Ali Aminian and Alex Xu. The book is the main study foundation for the case topics and many design alternatives in this handbook.

Recommended public attribution

This material began as personal study notes based largely on Machine Learning System Design Interview by Ali Aminian and Alex Xu. The restructuring, clarifications, trade-off discussions, diagrams, interactive questions, and additional interpretations are my own.

This project is not affiliated with the authors or publisher and is not a substitute for the original book.

Publication boundary

- Do not publish handwritten source scans.
- Do not reproduce book text, tables, or figures verbatim.
- Use original diagrams and wording for transformed explanations.
- Keep source-derived structure, confirmed interpretation, and added discussion visually distinct.